



USER MANUAL

SAAD FAROOQ
103498653@student.swin.edu.au

Contents

1.0 Introduction	2
2.0 Amazon Cognito	2
3.0 Route53 and Certificate	3
4.0 Load Balancer	5
5.0 Auto Scaling Group	7
6.0 Upload File to S3 Bucket	8

1.0 Introduction

In this document we would be discussing about the backend code develop on the AWS for the EngineRay web application. We have created a Cloud Development Kit by using typescript language where we create a stack. Moreover, we would discuss the functionality provided by each of the service and code explanation.

2.0 Amazon Cognito

Amazon Cognito is a service provided by the Amazon Web Services. The purpose of using this service is to provide Authentication user detail at the time of registration and login.

```
const EngineerUserPool = new cognito.UserPool(this, "enginerauserpool",{
  userPoolName:"enginera-userpool",
  selfSignUpEnabled:true,
  userVerification:{
    emailStyle:cognito.VerificationEmailStyle.CODE,
    emailSubject:"Engine Ray email verification",
    emailBody:
      "Hey user, Thank you for siging up with enginera here is your verification code {####}"
  },

  signInAliases: { email: true },
  autoVerify:{email: true},
  signInCaseSensitive:false,
  passwordPolicy:{
    minLength:8,
    requireDigits:true,
    requireLowercase:true,
    requireUppercase:true,
    requireSymbols:true,
  },
  standardAttributes:{
    fullname:{
      required:true,
    },
  },
  accountRecovery:cognito.AccountRecovery.EMAIL_ONLY,
  email: cognito.UserPoolEmail.withCognito("info@enginera.com"),
  removalPolicy: cdk.RemovalPolicy.DESTROY,
});
EngineerUserPool.grant(role,'cognito-idp:AdminCreateUser');
```

Figure 1: Code snapshot for Cognito

In the above snapshot we can analyse that first we are creating a user pool name EngineerUserPool. Then we are defining the attribute for the user pool as follow:

- **UserPoolName:** This attribute allows us to define the name of our userpool.
- **SelfSignedEnabled:** This attribute true shows that a user can register on our website without any need of granting a permission from an admin. Whereas, if the value of **Selfsigned** was false then whenever a user registered an admin had to approve his profile after which he would only be able to login on our portal.
- **userVerification:** Since the **selfSignedEnable** is true we used **userVerification** attribute to verify the user email address. So, whenever a user clicks on the register button it sends a autogenerated email with a unique code to the user to verify the email address

- **signInAlias**: In `signInAlias` we define the field which are required for the registration page of website. For instance, **email:true** reflect that a user needs to provide their email address. The **AutoVerify** if it's true we define which approach we are taking to for user verification for this project we are using email address but we can also opted to used phone number provided by the user. **PasswordPolicy** here we define required combination for a password. **Minlength** reflect that password provided by the user should have 8 characters. **RequireDigit** reflect that user must have a digit in their password combination. **Requirelowercase** and **requireuppercase** reflect that the user must have an uppercase character and a lowercase character in their password combination. And the **requireSymbol** reflect that a user needs to have at least one symbol in their password combination.
- **standardAttributes** we define the field name on the registration page. In our case we are using **fullname** field. But we can also define further fields like contact number. We can also define **customAttribute** which allow us to have more field like country and city on our registration page
- **Accountrecovery** here we define that in case if a user forgets their password how they can recover the access to their account in our case is through email.

3.0 Route53 and Certificate

The Route53 is a Domain Name Service provided by the Amazon Web Service. We have integrated this service so we can redirect the traffic to our webserver. Furthermore, Domain Certificate are used to allows HTTPS traffic for on our website for which we have also integrated Amazon Certificate Manager that validate the ownership of a domain certificates.

```
const domains = 'engineray.online';

const testzones = new route53.PublicHostedZone (this, 'HostedZone',{
  zoneName: domains,
});
const cer = new acm.Certificate(this, 'Certificate',{
  domainName: domains,
  validation: acm.CertificateValidation.fromDns(testzones),
});
const certarn = cer.certificateArn;
```

Figure2: Route53 and Certificate code snapshot

Now, we would be analysing the above code as follow:

- **Constdomain** here we are defining our domain name which in our case is `engineray.online` but we can also change the name as well.
- **Consttestzones** here we are creating the route53 hosted zone. We are using **publichostedzone** but it can also be change to **privatehostedzone** if required. In **privatehostedzone** we need to provide an extra attribute **VPC** that reflects that in which network our domain should work
- **zoneName** here we can define that which hosted zone records should be accessed when a user tries to our access our website.
- **Constcer** here we are creating a certificate. Here **this** is pointing towards the stack we are working on.
- **domainName** here we define the domain name for our certificates

- **validation** here we define the approach we want to use to validate our certificate in case we are using DNS Validation which is reflected by **fromDns** method and we are directing the validation method to our route53 zone we have created earlier. This validation method will verify the domain ownership through WhoIS directory. We can also use **fromemail** method where an email would be sent to the owner of the domain to validate the certificate.

```
testlistener.addTarget('testtargets',{
  targets:[new targets.InstanceTarget(testec2Instance)],
  port:80,
});
const testcon = new route53.ARecord(this,'Arecord', {
  zone: testzones,
  target: route53.RecordTarget.fromAlias(new target.LoadBalancerTarget(testloadbalancer)),
  //target: route53.RecordTarget.fromIpAddresses(testelasticIP.ref),
});
```

Figure 3: ARecord of Route53

We will analyse the code of above snapshot as follow:

- **const testcon** here we are creating a route53 ARecord.
- **Zone** here we define the zone we want to add our ARecord.
- **Target** here we define the target where we want to redirect the traffic. For our case we are redirecting the traffic to the Application Load Balancer. However, we can also add an attribute name domain where we can define the different wrong iteration of our domain. So, if a user enters a wrong domain it would still redirect to our Application Load balancer.

4.0 Load Balancer

The Application load balancer is a service provided by the Amazon Web Services. The purpose of this service is that we can redirect the HTTPS traffic on our webserver. It's can further divide the incoming load on our webserver to different webserver that are created by Auto Scaling Group. And it keeps checking the webserver by sending packets to identify the webserver health status. If a webserver responds to this packets, it's declare as Healthy and if a webserver is doesn't respond to the packet then it is declare as unhealthy and now the Auto Scaling Group has to launch another webserver. For the Application Load Balancer we need to further define a port it listen to and then we need to define a target where it redirect the traffic.

```
const Loadbalancersec = new ec2.SecurityGroup(this, 'loadbalSG',{
  vpc: vpc,
  allowAllOutbound: true,
  securityGroupName: 'LoadBalancerSG',
});
Loadbalancersec.addIngressRule(
  ec2.Peer.anyIpv4(),
  ec2.Port.tcp(80),
  "Allow HTTP traffic",
);
Loadbalancersec.addIngressRule(
  ec2.Peer.anyIpv4(),
  ec2.Port.tcp(443),
  "Allow HTTPS traffic",
);
Loadbalancersec.addEgressRule(
  testec2InstanceSG,
  ec2.Port.tcp(80)
);

const testloadbalancer = new elb2.ApplicationLoadBalancer(this, "apploadbalancer",{
  vpc: vpc,
  internetFacing: true,
  securityGroup: Loadbalancersec,
  // idleTimeout: Duration.seconds(60),
});
```

Figure 4: Application Load Balancer

Now we will analyse the above code as follow:

- **constLoadbalancersec** here we are creating a security group for our load balancer where we provide attribute like VPC that reflects that the load balancer must work in our created Virtual Private Cloud. Then we define the rules for our we have allow the outbound traffic by giving true value to **allowAllOutbound**. So, our load balancer should be able to communicate to our webserver.
- **addIngressRule** here we define the inbound rule for our load balancer in which we could define the open ports of our load balancer by using **Port** attributes and we can also define the traffic we want to accept by **anyIpv4**. So, the traffic coming from any IP Address would be accepted. In addition, we can also provide custom IP address as well. We have two inbound rules for this project. To allow HTTP and HTTPS traffic.
- **addEgressRule** here we define the outbound rule for our security group. For this project we have redirected the outgoing traffic to our webserver security on port 80. So, whether it's a HTTPS traffic or HTTP it would be redirected to port 80 of our webserver.
- **Consttestloadbalancer** here we are creating a Application Load Balancer for our project. Where define **VPC** we want the load balancer to initialize.

- **InternetFacing** here we define that the Load Balancer would be facing oncoming packets and then redirect them to the webserver.

```
const testlistener = testloadbalancer.addListener('testlist',{
  port:443,
  certificates:[cer],
  protocol: elb2.ApplicationProtocol.HTTPS,
});
```

Figure 5: Load Balancer Listener

- **Consttestlistener** here we are creating a listener for our load balancer so it's listen to the traffic particular. As in the attributes we have define that it should listen to port 443. And only the HTTPS request. We also attached the certificate to our load balancer in the Certificates attributes.

```
testlistener.addTarget('testtargets',{
  targets:[new targets.InstanceTarget(testec2Instance)],
  port:80,
});
const testcon = new route53.ARecord(this,'Arecord', {
  zone: testzones,
  target: route53.RecordTarget.fromAlias(new target.LoadBalancerTarget(testloadbalancer)),
  //target: route53.RecordTarget.fromIpAddresses(testelasticIP.ref),
});
```

Figure 6: Load Balancer Target

- **addTarget** now we are defining that when a listener receives a packet where it should be redirected. For this project we are redirecting it to our webserver which is an EC2 instance by defining it in the **targets** attribute. However, we could also redirect it to the Auto Scaling Group. In addition, we are also defining the **port** attribute in which we define that to which port our listener traffic should be redirected.

5.0 Auto Scaling Group

The Auto Scaling Group is a service that provided by the Amazon Web Service. That can launch multiple web server if the primary webserver is taking time to response. So, that we could avoid delays and also if the no request is greater then what the webserver could handle.

```
const ami = ec2.MachineImage.genericLinux({
  "us-east-2": "ami-0189375cbeafabd08"
})
const testautoscale = new autoscaling.AutoScalingGroup(this, 'ASG', {
  vpc,
  instanceType: ec2.InstanceType.of(ec2.InstanceClass.T2, ec2.InstanceSize.MICRO),
  vpcSubnets: {
    subnetType: ec2.SubnetType.PUBLIC,
  },
  machineImage: ami,
  associatePublicIpAddress: true
});
testautoscale.applyRemovalPolicy(RemovalPolicy.DESTROY);
```

Figure 7: Auto Scaling Group

Now we will analyse the above code snapshot,

- First, we are defining a Machine Image in the **constami** where we define that which Image of the EC2 instance should be used to launch more instances. An Amazon Machine Image has the configuration of the primary webserver.
- Now we define the **VPC** it should launch instances in.
- **InstanceType** here we define the computational configuration of the instance.
- **Vpcsubnet** here we define that in which subnet the Auto Scaling Group should launch an instance. For this project we are allowing it to launch an EC2 instance (webserver) in the public subnet
- **associatePublicIpAddress** here we are associating a public IP address with all the EC2 instance(webserver) that would be launch by the Auto Scaling Group.

6.0 Upload File to S3 Bucket

For this project we have also written a PHP code that's responsible to upload files into our s3 bucket. The file could be either an image or any other document. In the code snapshot below we are using s3 client library to connect our PHP code with Amazon Web Services environment. In this we need to provide the region of the service we wish to connect and also provide the IAM user credential so that the code could access the S3 bucket. Then we are using s3->putobject method to upload our file directly to the S3 bucket.

```
<?php
if(isset($_FILES['image'])) {
    $file_name = $_FILES['image']['name'];
    $temp_file_location = $_FILES['image']['tmp_name'];

    require 'aws-autoloader.php';
    // use \Aws\S3\S3Client;
    // use \Aws\S3\Exception\S3Exception;

    $s3 = new Aws\S3\S3Client([
        'region' => 'us-east-2' ,
        'version' => 'latest',
        'credentials' => [
            'key' => "AKIAUTEXLE6CKXRRL4N4",
            'secret' => "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",
        ]
    ]);

    $result = $s3->putObject([
        'Bucket' => 'enginerayprojectstack-engineraybucket2b983bf9-9flhu9hi3w15',
        'Key' => $file_name,
        'SourceFile' => $temp_file_location
    ]);
    $object_id = $temp_file_location;
    echo $object_id;
    //var_dump($result);
}

?>

<form action="<?=$_SERVER['PHP_SELF']; ?>" method="POST" enctype="multipart/form-data">
    <input type="file" name="image" />
    <input type="submit"/>
</form>
```

Figure 8: PHP code to upload to S3 bucket